SCRIPTING GUIDE

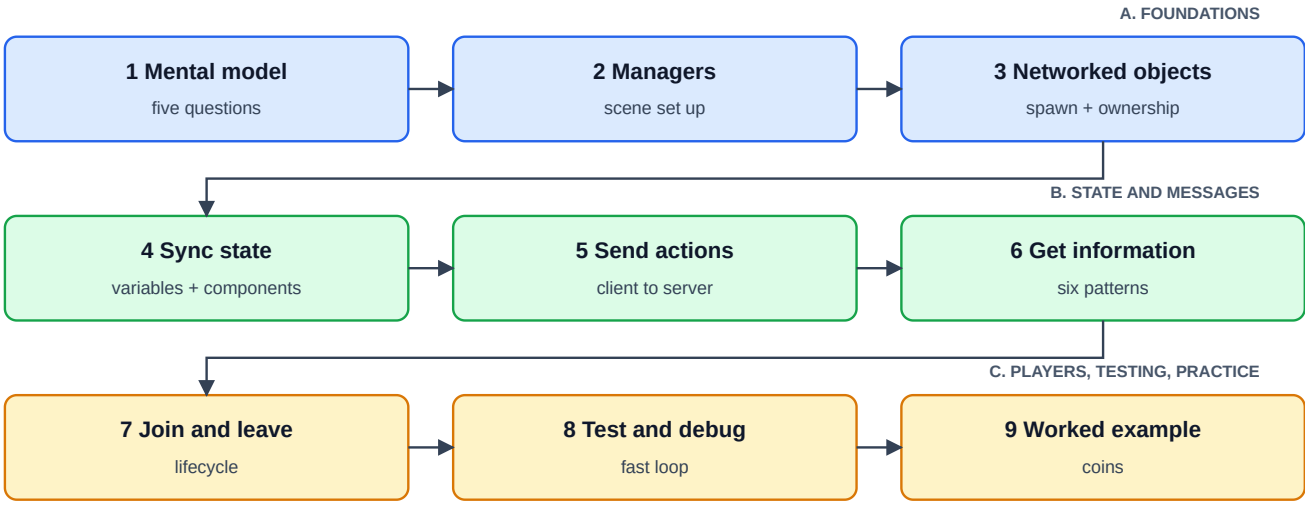
Writing multiplayer scripts: a step-by-step guide

Netcode for GameObjects in our workflow, explained for people who know Mirror

> **How this guide is built.** Nine steps in one order. Each step says what it **needs** from earlier steps and what it **gives** to later ones, shows where you are, and ends with a check. It is not an API list: every tool appears at the moment you need it.

Repository: github.com/SynokJ/multiplayer_maze | Unity 6 | Netcode for GameObjects 2.13.3 | API names checked in the package source

The road in one picture



Each step uses what the previous one built. Step 9 uses all of them once.

Facts about our workflow that shape every script.

- The **server is a Linux program** started by the lobby, one per room. It has no player. `IsHost` is never true in a real game.
- **Every player is a WebGL client.** A browser cannot listen for connections, so no player can be the server.
- **All clients talk only to the server.** Two clients never talk to each other directly.
- **The server decides.** Clients send requests; the server changes the state and tells everyone.

How the steps are connected

Each step produces something that later steps use. If a step does not make sense, look at what it needed.

Step	Name	What it hands on	Used in step
1	**Mental model**	The five questions; the Mirror to NGO translation	2-9
2	**Managers**	A scene with NetworkManager, transport, prefab list and your own manager object	3, 7, 9
3	**Networked objects**	A prefab that can be spawned, owned and despawned by the server	4, 5, 9
4	**Sync state**	NetworkVariables and synced components that show the same state on every screen	5, 6, 9
5	**Send actions**	Rpc methods that let a client ask the server to do something	6, 9
6	**Get information**	Six patterns for asking and answering across the network	9
7	**Join and leave**	Per-player data that appears and disappears with connections	9
8	**Test and debug**	A quick Editor loop and a table of error messages	9
9	**Worked example**	Coins: spawn, touch, score, display. Every step used once.	-

A FOUNDATIONS

STEP 1 of 9 The mental model: five questions

1 2 3 4 5 6 7 8 9

A foundations | B state and messages | C players, test, practice

Why this step: Every multiplayer feature answers the same five questions. Each Netcode tool is the answer to one of them, so if you know the question you know which tool to reach for.

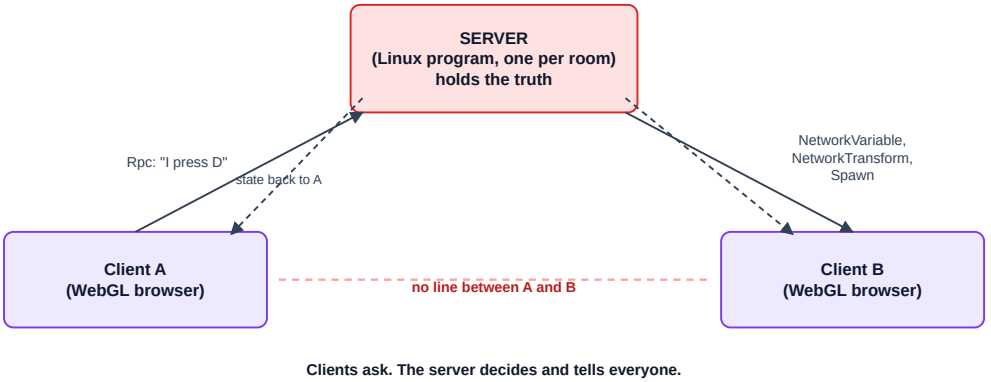
NEEDS

- Nothing. This is the start.

→ THIS STEP →

GIVES

- The five questions and the Mirror-to-NGO translation table. Every later step answers one question.



Who talks to whom

Do

#	Question	Answer in Netcode for GameObjects (NGO)	Step
1	**When does the object exist?**	The server **spawns** it; it **despawns** it	3
2	**Who may change it?**	One owner of the truth: the **server** (default) or the **owner** client	3, 4
3	**How do the others see it?**	**NetworkVariable** (values), **NetworkTransform** and **NetworkAnimator** (components)	4
4	**How does a client ask for a change?**	An **Rpc** sent to the server	5
5	**How do I get information?**	Read a NetworkVariable, or **ask with an Rpc and get an Rpc back**	6

If you know Mirror, this table translates your habits. The NGO column was checked against the NGO 2.13.3 source code.

Mirror	NGO (our workflow)	What to remember
`NetworkIdentity`	`NetworkObject`	One per networked prefab, on the root
`NetworkBehaviour`	`NetworkBehaviour`	Same name. Scripts with synced state must inherit it

Mirror	NGO (our workflow)	What to remember
`[SyncVar]`	`NetworkVariable<T>`	A field you declare and set through `.Value`. Only the server may write it by default
SyncVar `hook`	`OnValueChanged(previous, next)`	Subscribe in `OnNetworkSpawn`
`SyncList<T>`	`NetworkList<T>`	`T` must be `unmanaged` and `IEquatable<T>`
`[Command]`	`[Rpc(SendTo.Server)]`	**Not owner-only by default.** Add `InvokePermission = RpcInvokePermission.Owner`
`[ClientRpc]`	`[Rpc(SendTo.ClientsAndHost)]`	On a dedicated server this reaches every client
`[TargetRpc]`	`[Rpc(SendTo.SpecifiedInParams)]` with `RpcTarget.Single(...)`, or `SendTo.Owner`	Step 6
`[Server]` / `[Client]`	`if (!IsServer) return;`	A normal `if`, not an attribute
`isServer`, `isClient`, `isLocalPlayer`, `isOwned`	`IsServer`, `IsClient`, `IsLocalPlayer`, `IsOwner`	Valid only after spawn
`OnStartServer`, `OnStartClient`, `OnStartLocalPlayer`	`OnNetworkSpawn()`	One callback; check `IsServer` and so on inside it
`OnStopServer`, `OnStopClient`	`OnNetworkDespawn()`	Unsubscribe here
`NetworkServer.Spawn(go)`	`networkObject.Spawn()`	Server only
`NetworkServer.Spawn(go, conn)`	`networkObject.SpawnWithOwnership(clientId)`	
`NetworkServer.Destroy(go)`	`networkObject.Despawn()`	
`conn.connectionId`	`ulong clientId` (`OwnerClientId`, `SenderClientId`)	The server's own id is 0
client-authority `NetworkTransform`	`NetworkTransform` with `AuthorityMode = Owner`	Default is `Server`
`Authenticator`	`NetworkManager.ConnectionApprovalCallback`	Our lobby token is checked here
method name `CmdX`, `RpcX`	method name **must end in `Rpc`**	Otherwise the build fails

The Mirror column is from general Mirror knowledge, not re-checked in this work.

> **CHECK IT WORKED**
> Pick any feature you want, for example "a coin disappears when touched". You can say who decides, which tool shows it to everyone, and which tool carries a request.

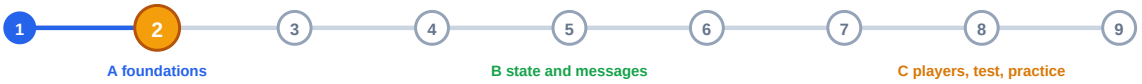
WATCH OUT

Mistakes this step prevents: treating a player as the host (there is none); writing `MonoBehaviour` where `NetworkBehaviour` is needed; expecting `[Rpc(SendTo.Server)]` to reject non-owners without `InvokePermission`.

NEXT → Step 2 sets up the scene objects that make all five answers possible.

A FOUNDATIONS

STEP 2 of 9 Set up the managers



Why this step: Nothing in this guide works until one NetworkManager exists, knows its transport and knows which prefabs may be spawned. This is the one-time setup; later steps only add your own scripts on top.

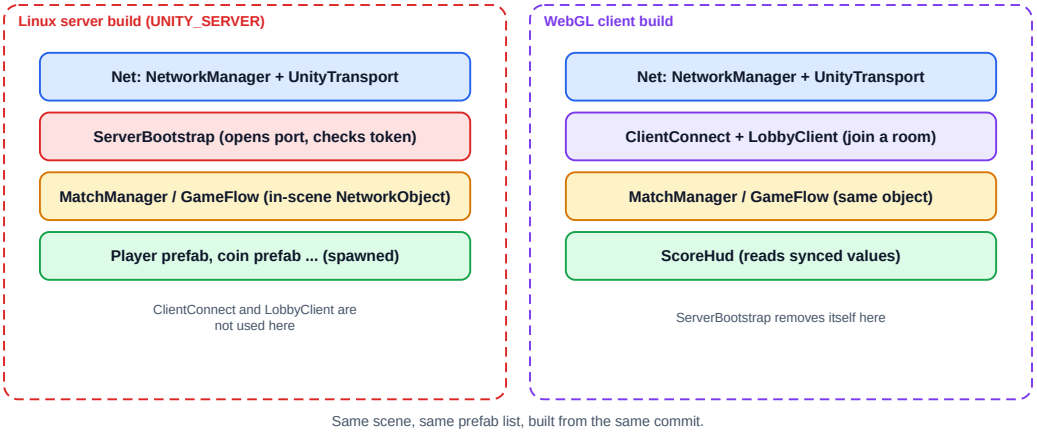
NEEDS

- Step 1: the five questions.
- The tutorial (Step 8): the `Net` object with NetworkManager, UnityTransport and the scripts from `unity-scripts/`.

→ THIS STEP →

GIVES

- A scene where the server starts, clients connect, and your own manager object holds the match state.



The same scene in two builds

Do

2.1 What lives on the `Net` object. One object, several scripts. Each script runs only where it makes sense.

Script	Runs in	Job
`NetworkManager` + `UnityTransport`	server and client	The connection itself. **Exactly one** per game
`ServerBootstrap`	Linux server only	Reads `-port`, starts the server, checks join tokens, reports to the lobby
`ClientConnect`, `LobbyClient`	WebGL client only	Ask the lobby for a room, then connect
`ScoreHud`	client	Shows synced values (Step 9)
`DevHost`	Editor only	Press H to host inside the Editor for fast tests (Step 8)

`ServerBootstrap` removes itself in builds that are not the server, so the same scene works everywhere.

2.2 The NetworkManager settings that matter. Defaults below were read in `NetworkConfig.cs` (NGO 2.13.3).

Setting	Default	What it means for you
Player Prefab	none	The object every connecting player gets. Our server asks for it in `ConnectionApproval` (`CreatePlayerObject = true`)
Network Prefabs Lists	empty	**Every prefab you spawn at runtime must be in this list**, or spawning fails
Tick Rate	`30`	Network state goes out 30 times a second. Higher costs bandwidth and CPU on the VM
Connection Approval	off	The scripts switch it on. Needed for the lobby token
Enable Scene Management	on	Allows network scene switches and fixes scene differences when a client connects (tooltip in the source)
Force Same Prefabs	on	Checks that client and server have the **same prefab list** when connecting. This is why both builds come from one commit. Turn it off only if you add prefabs at runtime
Spawn Timeout	`10` s	How long a message is held if the object it is for has not spawned yet; after that it is dropped
Client Connection Buffer Timeout	`10` s	Server side: how long the server waits for the approval handshake before it disconnects the client

2.3 Events you will subscribe to. Names read in `NetworkManager.cs`. Subscribe in `Start` or `OnNetworkSpawn`, and unsubscribe when done.

Event	Fires	Use it for
`OnServerStarted` / `OnClientStarted`	when this process starts listening / connecting	Start game logic
`OnConnectionEvent`	on connection changes. Event types `ClientConnected`, `PeerConnected`, `ClientDisconnected`, `PeerDisconnected`	**Preferred.** The source says the older callbacks "will eventually be deprecated". `PeerConnected` tells other clients that someone joined
`OnClientConnectedCallback(ulong)` / `OnClientDisconnectCallback(ulong)`	on the server for every client, and on a client for itself only	Simple cases; our current scripts use these
`OnServerStopped(bool)` / `OnClientStopped(bool)`	after shutdown	Clean up
`DisconnectReason` (string)	after a refused or ended connection	Shows the `Reason` text from approval

2.4 Your own manager. Put match-wide state in one **in-scene NetworkObject** with a `NetworkBehaviour` (the repository has `GameFlow`). It holds NetworkVariables; only the server changes them.

```
public class MatchManager : NetworkBehaviour
{
    public static MatchManager Instance { get; private set; }
    public NetworkVariable<GamePhase> Phase = new NetworkVariable<GamePhase>(GamePhase.Lobby);

    public override void OnNetworkSpawn()   { Instance = this; }        // set here, not in Awake:
network flags are valid now
```

```
    public override void OnNetworkDespawn() { if (Instance == this) Instance = null; }

    public void StartMatch() { if (IsServer) Phase.Value = GamePhase.Playing; }   // the server
writes; everyone reads
}
```

2.5 Server-only code. Two tools, two uses:

- **Run time:** `if (!IsServer) return;` inside gameplay scripts. It also works when you test in the Editor as a host.
- **Build time:** `#if UNITY_SERVER` only for the bootstrap that must not exist in the WebGL build.

CHECK IT WORKED

Press Play in the Editor. The Console has no red errors and no "NetworkPrefab not registered" messages. After you add `DevHost` to `Net` and press H, your player appears.

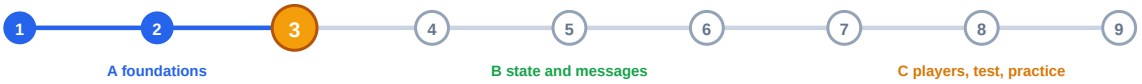
WATCH OUT

Common mistakes: two NetworkManagers in the scene; a runtime-spawned prefab missing from the list; different prefab lists in the client and server builds (build both from one commit); setting `Instance` in `Awake`.

NEXT → Step 3 turns your own prefabs into objects the manager can spawn.

A FOUNDATIONS

STEP 3 of 9 Make an object networked



Why this step: A normal GameObject exists only on one machine. A **NetworkObject** is the passport that lets the server create the same object on every client, track who owns it, and remove it again.

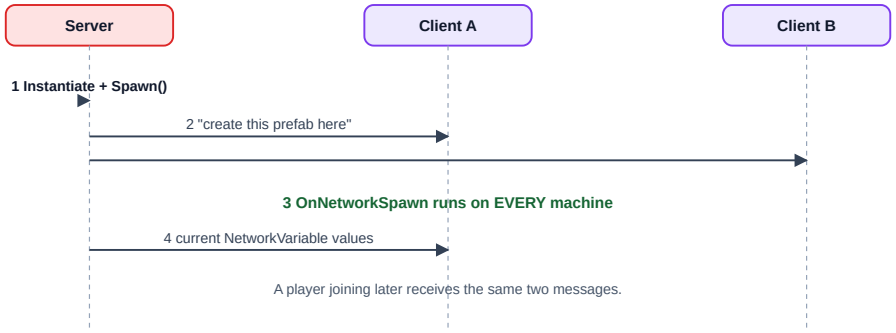
NEEDS

- Step 2: a NetworkManager with a Network Prefabs List.

→ THIS STEP →

GIVES

- A prefab the server can spawn, give to a player and despawn. Steps 4 and 5 add state and actions to it.



What spawning does

Do

3.1 The prefab checklist.

1. Add a **NetworkObject** to the root of the prefab.
2. Scripts that have synced state or Rpcs inherit `NetworkBehaviour`, not `MonoBehaviour`.
3. Put the prefab into the **Network Prefabs List** that the NetworkManager uses.
4. Spawn it **from the server**. A client cannot spawn.

3.2 Spawn, own, despawn. The signatures were read in `NetworkObject.cs`.

```
// SERVER ONLY. Everyone gets a copy; OnNetworkSpawn runs on every machine.
var no = NetworkObject.InstantiateAndSpawn(prefab, NetworkManager, position: pos, rotation:
Quaternion.identity);

// Same thing in two lines:
var go = Instantiate(prefab, pos, Quaternion.identity);
go.GetComponent<NetworkObject>().Spawn();                          // owner = the server

go.GetComponent<NetworkObject>().SpawnWithOwnership(clientId);  // owned by that player
networkObject.ChangeOwnership(newClientId);                       // hand it over
networkObject.RemoveOwnership();                                  // back to the server
networkObject.Despawn();                                          // gone on every machine; true by
default also destroys it
```

Two details found in the source: `InstantiateAndSpawn` has `rotation = default`, and `default` for a rotation is an **all-zero quaternion**, so always pass `Quaternion.identity`. It also logs an error if the prefab is not registered or if you are not the server.

3.3 Players are objects too. Our server approves a connection with `CreatePlayerObject = true`. NGO then spawns the **Player Prefab** and gives the client ownership. On the owner's machine `IsOwner` and `IsLocalPlayer` are true. The player object is found with `NetworkManager.LocalClient.PlayerObject` on a client, or `NetworkManager.SpawnManager.GetPlayerNetworkObject(clientId)`.

3.4 What `Is...` means. Valid after spawn only.

Flag	True when
`IsServer`	this process is the server (the Linux room)
`IsClient`	this process is a client (the browser)
`IsOwner`	this machine owns this object (the player's own player, or the server for server-owned objects)
`IsLocalPlayer`	this is the player object that belongs to this machine
`IsOwnedByServer`	the server owns it, for example a coin
`IsSpawned`	the object exists on the network now

3.5 Where code goes: the lifecycle.

Method	Use it for	Do not
`Awake`	Creating `NetworkList` s	Read `IsServer` or `IsOwner`: not set yet
`OnNetworkSpawn`	Subscribe to events, initialise state, first values	
`OnNetworkDespawn`	Unsubscribe, clean up	
`OnGainedOwnership` / `OnLostOwnership`	React to a changed owner	

3.6 Objects that are already in the scene. A NetworkObject placed in the scene is spawned for you when the scene loads on the server. `OnInSceneObjectsSpawned` tells you when they all exist. To show an object to only some players use `NetworkShow(clientId)` and `NetworkHide(clientId)`.

3.7 When the owner leaves. The object is destroyed unless **Don't Destroy With Owner** is ticked on its NetworkObject (default off; read in the source tooltip).

CHECK IT WORKED

In the Editor with `DevHost` (press H): a test button or script spawns your prefab and it appears. In the Hierarchy it shows as a clone with a NetworkObject.

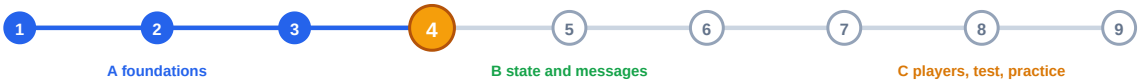
WATCH OUT

Common mistakes: spawning from a client (nothing happens or an error is logged); forgetting the prefab list; calling an Rpc before the object is spawned (see Step 8); using `default` rotation.

NEXT → Step 4 gives the spawned object values and components that stay the same everywhere.

B STATE AND MESSAGES

STEP 4 of 9 Sync state to everyone



Why this step: Spawning creates the object everywhere, but nothing else is copied. Health, score, colour, position: each is synced only if you say so. This is the same rule as in Mirror: only declared values travel.

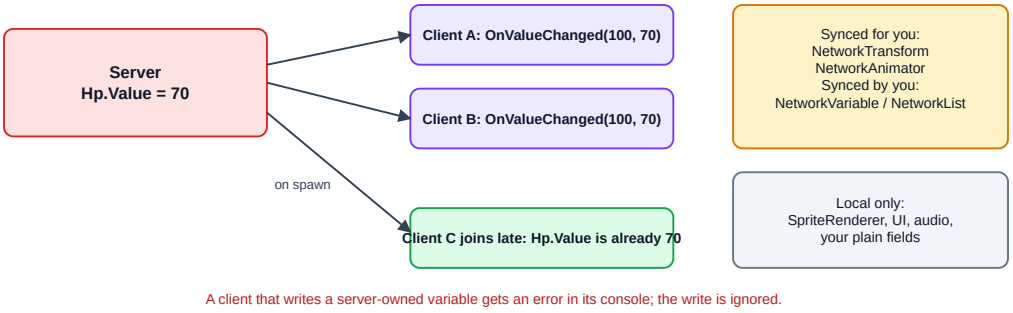
NEEDS

- Step 3: a spawned NetworkObject with a NetworkBehaviour.

→ THIS STEP →

GIVES

- Values and components that are the same on every screen, including for players who join late.



Synced, local, and late joiners

Do

4.1 NetworkVariable: the SyncVar of NGO. Source facts: the constructor is `NetworkVariable<T>(value, readPerm = Everyone, writePerm = Server)`, `OnValueChanged(previous, next)` is its change event, and assigning `.Value` marks it dirty and sends it.

```
public class Health : NetworkBehaviour
{
    public NetworkVariable<int> Hp = new NetworkVariable<int>(100);   // server writes, everyone reads

    public override void OnNetworkSpawn()
    {
        Hp.OnValueChanged += OnHpChanged;       // runs on every machine when the value changes
        OnHpChanged(0, Hp.Value);               // show the current value once; late joiners get theirs this way
    }
    public override void OnNetworkDespawn() { Hp.OnValueChanged -= OnHpChanged; }

    void OnHpChanged(int previous, int current) { /* update the health bar */ }

    public void Damage(int n) { if (IsServer) Hp.Value = Mathf.Max(0, Hp.Value - n); }
}
```

Rule	Why
Write only on the server	A client that writes a server-owned variable gets an error `Write permissions (Server) for this client instance is not allowed!` in its console, and the write is ignored (read in the source). It does not throw
Initialise as a field, never `null`	The variable must exist before spawn
Subscribe in `OnNetworkSpawn`, unsubscribe in `OnNetworkDespawn`	Prevents duplicate handlers after respawn
Use `FixedString32Bytes` (or 64, 128) for text	A normal `string` is not a plain value type
Use enums, numbers, `Vector2/3`, `Color` and small structs	Plain value types sync
Custom struct	Implement `INetworkSerializable` (an example, `ScoreEntry`, is in `GameFlow.cs`)
Late joiners	They receive the **current** value when the object spawns for them; you do not resend

Who may write or read. Pass the permissions in the constructor:

```
// The owner writes its own aim direction; everyone reads it. Trust it only for cosmetic things.
public NetworkVariable<Vector2> Aim = new NetworkVariable<Vector2>(default,
    NetworkVariableReadPermission.Everyone, NetworkVariableWritePermission.Owner);
```

Lists. `NetworkList<T>` needs `unmanaged` and `IEquatable<T>`, is created in `Awake`, changed on the server, and raises `OnListChanged`.

4.2 Components: what is synced for you, and what is not.

Component	Syncs	Note
NetworkTransform	position, rotation, scale	Tick the axes you need. For a 2D game turn off Z position and X/Y rotation. `AuthorityMode` is **Server** by default; **Owner** lets the owner move it. Has `Interpolate` (on by default) and a position threshold (`0.001` default)
NetworkAnimator	Animator parameters and triggers	Same two authority modes. Set triggers on the authority side
NetworkRigidbody2D	helps Rigidbody2D objects follow the NetworkTransform	Use with NetworkTransform
`SpriteRenderer`, `Text`, `AudioSource`, your own fields	**Nothing**	They are local. Drive them from a NetworkVariable's `OnValueChanged`

So "syncing part of a component" means: choose which NetworkTransform axes to send, and keep everything else in a NetworkVariable that your script applies to the component.

```
// Colour is not synced by SpriteRenderer. A NetworkVariable carries it, a handler applies it.
public NetworkVariable<Color> Tint = new NetworkVariable<Color>(Color.white);
public override void OnNetworkSpawn()
{
```

```
    Tint.OnValueChanged += (_, c) => GetComponent<SpriteRenderer>().color = c;
    GetComponent<SpriteRenderer>().color = Tint.Value;
}
```

4.3 Cost. NetworkVariables send only when changed. NetworkTransform sends when the value moves past its threshold. Sync what others need to see, not what only the server needs.

CHECK IT WORKED

Host in the Editor (press H) with `PlayerState`. Change `Score` from a test button on the server side: the log line `... score 0->1` appears. Then join with a second client later and see the current score immediately.

WATCH OUT

Common mistakes: writing a variable on a client (error in the console); creating a `NetworkList` in `OnNetworkSpawn`; forgetting to unsubscribe; expecting the colour of a sprite to sync by itself; syncing a `string`.

NEXT → Step 5 lets clients change things, by asking the server.

B STATE AND MESSAGES

STEP 5 of 9 Send actions to the server

1 2 3 4 **5** 6 7 8 9

A foundations | B state and messages | C players, test, practice

Why this step: Clients cannot write server-owned state. They send a request, an **Rpc** (remote procedure call), and the server decides. This is the NGO version of `[Command]`.

NEEDS

- Step 4: state the server can change.
- Step 3: the Rpc lives on a **spawned** NetworkBehaviour.

→ THIS STEP →

GIVES

- Methods a client can call that run on the server, with the sender identified and the request checked.

Do

5.1 Anatomy of an Rpc. Every fact here was read in `RpcAttributes.cs`, `RpcTarget.cs` and `NetworkBehaviour.cs`.

```
[Rpc(SendTo.Server, InvokePermission = RpcInvokePermission.Owner)]     // who receives, who may call
void FireRpc(Vector2 direction, RpcParams rpcParams = default)         // name ends in Rpc; RpcParams last
{
    ulong sender = rpcParams.Receive.SenderClientId;                   // who really sent it
    // ... validate ... then do the work on the server
}
```

Part	Rule
Method name	**Must end with `Rpc`**, or the build fails. The method must not be `static` or generic (also build errors). It may be private
Where	A method of a `NetworkBehaviour` on a **spawned** object. Calling earlier throws `RpcException: The NetworkBehaviour must be spawned before calling this method`
`SendTo.Server`	Runs on the server. If you call it on the server it simply runs locally
`InvokePermission`	`Everyone` (**default**), `Server`, or `Owner`. With `Owner`, a non-owner calling it gets `This RPC can only be sent by its owner`
`Delivery`	`Reliable` (default) or `Unreliable` for things like frequent aim updates
Parameters	Numbers, enums, vectors, `FixedString`, `INetworkSerializable`, `NetworkObjectReference`. No return value
`RpcParams`	Last parameter, default `= default`. `rpcParams.Receive.SenderClientId` is the sender

5.2 Always check on the server. The default permission is **Everyone**. Mirror's `[Command]` is owner-only unless you say otherwise; this is the opposite, so set `Owner` yourself where it matters, and still validate:

```
float lastFire;
[Rpc(SendTo.Server, InvokePermission = RpcInvokePermission.Owner)]
void FireRpc(Vector2 dir)
```

```
{
    if (Time.time - lastFire < 0.3f) return;                          // rate limit
    if (dir.sqrMagnitude < 0.01f || dir.sqrMagnitude > 1.01f) return; // sane value
    lastFire = Time.time;
    SpawnBullet(dir.normalized);                                       // the server does the work
}
```

5.3 Send on change, not every frame. `MoverNet` sends the input only when it changes. The server keeps moving the player from the last input.

Do	Do not
Send intent: "I press D"	Send results: "I am at x = 5"
Clamp and test every value	Trust a position, score or damage from a client
Use `Unreliable` for repeatable data	Send an Rpc every frame

CHECK IT WORKED

In host mode (Step 8) call your Rpc from the owner: the server log shows it ran. Call it from a non-owner: you see `This RPC can only be sent by its owner.`

WATCH OUT

Common mistakes: method name without the `Rpc` suffix; calling in `Awake` before spawn; leaving the default `Everyone` on an owner-only action; trusting client values.

NEXT → Step 6 covers the other direction: how information comes back.

STEP 6 of 9 Get information from the server and from others



Why this step: An Rpc has no return value, and clients never talk directly. So "give me information" always becomes either a read of synced state, or two messages: a question and an answer. There are six patterns.

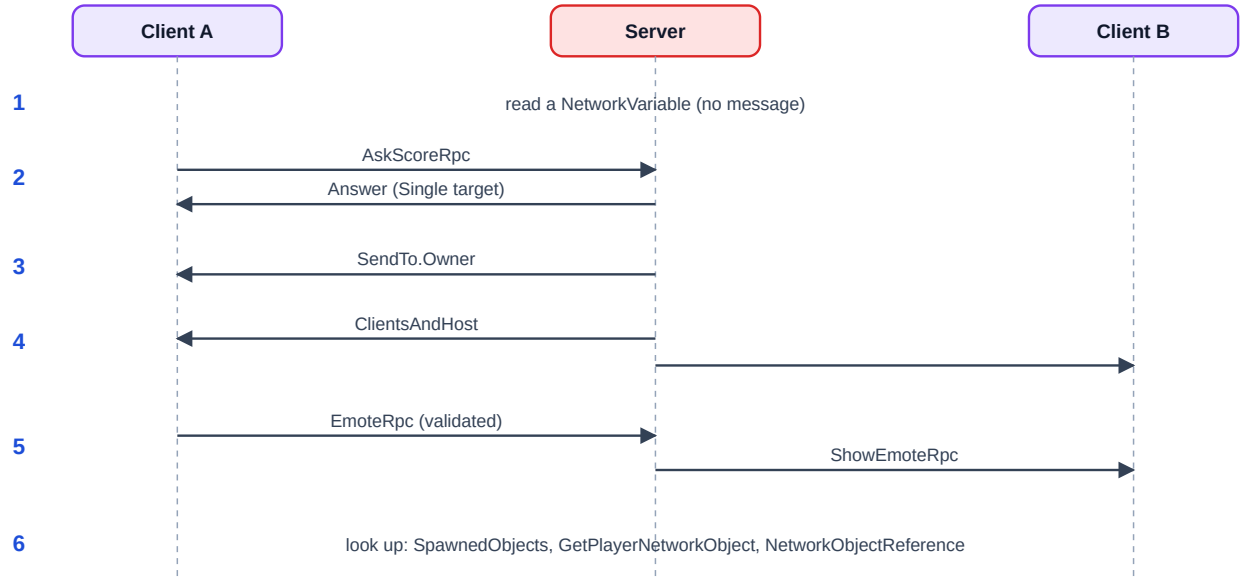
NEEDS

- Step 4: NetworkVariables.
- Step 5: Rpc basics.

→ THIS STEP →

GIVES

- The right pattern for each kind of information: shared, private, broadcast, client to client, lookups.



The six ways to get information

Do

Which pattern?

#	I want to ...	Pattern
1	Read something everyone may know (score, phase)	**Read a NetworkVariable.** No call needed
2	Ask the server something and get a private answer	**Request Rpc, then reply Rpc to that client**
3	Tell one player something private	Server Rpc with `SendTo.Owner` or a single target
4	Tell all players an event happened	`[Rpc(SendTo.ClientsAndHost)]`
5	Tell the other players something a player did	Client to server Rpc, **server validates, then broadcasts**
6	Look up another object or player	`SpawnManager`, `NetworkObjectReference`

Pattern 1: shared state. `var s = player.Score.Value;` on any machine. This is the cheapest and the best answer to "call info from the server".

Pattern 2: ask and answer. The question goes to the server; the answer goes only to the asker. The target helper `RpcTarget.Single(clientId, RpcTargetUse.Temp)` was read in `RpcTarget.cs`; passing it as the last argument works because `RpcParams` converts from a target.

```
[Rpc(SendTo.Server)]
public void AskScoreRpc(RpcParams rpcParams = default)
{
    ulong asker = rpcParams.Receive.SenderClientId;                    // from the connection,
never from a parameter
    AnswerScoreRpc(Score.Value, RpcTarget.Single(asker, RpcTargetUse.Temp));
}

[Rpc(SendTo.SpecifiedInParams)]                                        // needs a target at the
call site
void AnswerScoreRpc(int score, RpcParams rpcParams = default) { Debug.Log($"my score is {score}");
}
```

Pattern 3: private message. `[Rpc(SendTo.Owner)] void YouFoundAnItemRpc(int itemId)` called by the server runs on the machine of the object's owner. This is Mirror's `[TargetRpc]` for the owner.

Pattern 4: broadcast event. `[Rpc(SendTo.ClientsAndHost)] void ExplodedRpc(Vector3 pos)`. On a dedicated server this reaches every client. Beware: `SendTo.NotServer` does the same on the dedicated server but would skip a host in the Editor.

Pattern 5: client to other clients. Clients have no direct line. The reliable way is two hops, so your checks run:

```
[Rpc(SendTo.Server)]
void EmoteRpc(int emoteId, RpcParams rpcParams = default)
{
    if (emoteId < 0 || emoteId > 7) return;                            // validate
    ShowEmoteRpc(rpcParams.Receive.SenderClientId, emoteId);           // then tell everyone
}
[Rpc(SendTo.ClientsAndHost)] void ShowEmoteRpc(ulong from, int emoteId) { /* play the animation */
}
```

(The source shows NGO can also forward a client's message to other clients itself, using a proxy through the server. Do not use that for anything that needs checking.)

Pattern 6: find other things.

Need	Call
All connected players (server)	`NetworkManager.ConnectedClientsIds`
One player's object	`NetworkManager.SpawnManager.GetPlayerNetworkObject(clientId)`
My own player (client)	`NetworkManager.LocalClient.PlayerObject`
Any spawned object by id	`NetworkManager.SpawnManager.SpawnedObjects[id]`
Pass an object in an Rpc	a `NetworkObjectReference` parameter. It converts from a `NetworkObject` and back, and has `TryGet`
Server time shared by all	`NetworkManager.ServerTime.Time`

```
[Rpc(SendTo.Server)]
void PickUpRpc(NetworkObjectReference item, RpcParams rpcParams = default)
```

```
{
    if (!item.TryGet(out NetworkObject obj)) return;                    // the object may already
be gone
    // ... check distance, ownership, and so on, then act
}
```

CHECK IT WORKED

You can name the pattern for each of these: a scoreboard, a private "you won" message, an explosion sound for all, a chat line, "which player is nearest".

WATCH OUT

Common mistakes: trying to return a value from an Rpc; sending a client id as a parameter and trusting it (use `SenderClientId`); forgetting that `SendTo.SpecifiedInParams` needs a target at the call; using the same pattern for private and public information.

NEXT → Step 7 follows these messages through a player joining and leaving.

C PLAYERS, TESTING, PRACTICE

STEP 7 of 9 Players joining and leaving



Why this step: Everything so far assumed the players are already there. Real rooms fill and empty. Your scripts must set up per-player data on arrival and clean it up on departure, and deal with the browser dropping connections.

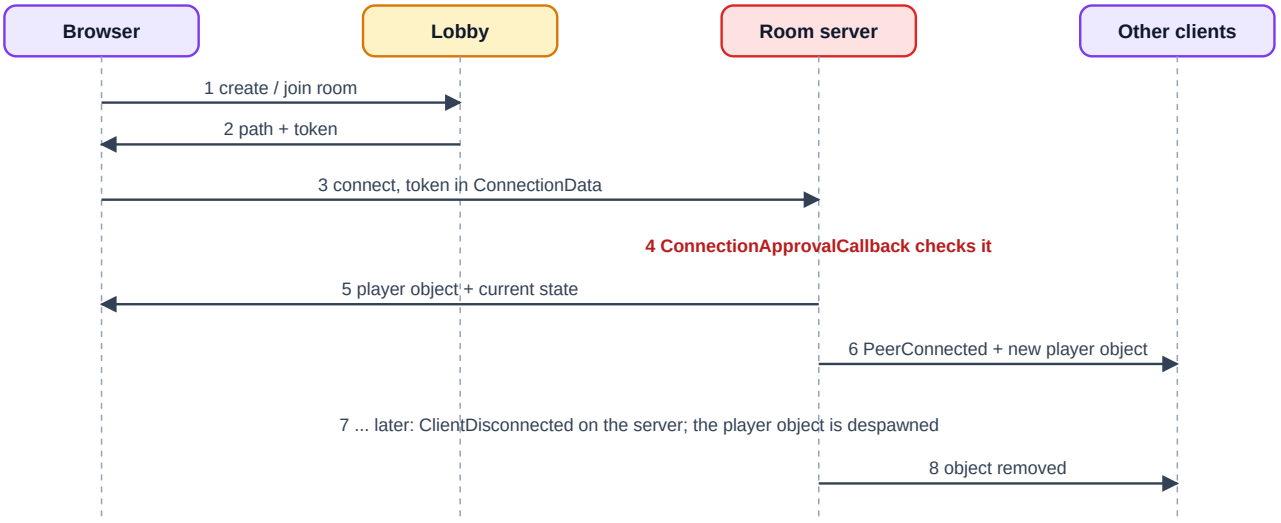
NEEDS

- Step 2: connection events.
- Step 3: player objects.
- Step 6: how to find players.

→ THIS STEP →

GIVES

- A tidy join/leave routine on the server and a reconnect rule for the client.



A player joins

Do

7.1 What happens when a player joins in our workflow.

1. The browser asks the lobby for a room and gets a **token**.
2. `ClientConnect` puts the token in `NetworkConfig.ConnectionData` and starts the client.
3. On the server, `ConnectionApprovalCallback` checks the token. It sets `Approved`, `CreatePlayerObject` and a `Reason` for refusals.
4. NGO spawns the player object and sends the new client every object that exists and the **current** value of every NetworkVariable.
5. `OnNetworkSpawn` runs on the new client for all those objects. The other clients get `PeerConnected` and the new player object.

7.2 Per-player data on the server. Keep a dictionary and fill it from the connection event:

```
class PlayerData { public int Kills; }
readonly Dictionary<ulong, PlayerData> players = new();

public override void OnNetworkSpawn()   { if (IsServer) NetworkManager.OnConnectionEvent +=
OnConn; }
public override void OnNetworkDespawn() { if (IsServer) NetworkManager.OnConnectionEvent -=
OnConn; }

void OnConn(NetworkManager nm, ConnectionEventData e)
{
    if (e.EventType == ConnectionEvent.ClientConnected)    players[e.ClientId] = new PlayerData();
    if (e.EventType == ConnectionEvent.ClientDisconnected) players.Remove(e.ClientId);
}
```

On the server the event `ClientConnected` and `ClientDisconnected` carry the id of the player that came or left. On a client the same events refer to the client itself.

7.3 When a player leaves. The player object is despawned. Objects it owned are destroyed too, unless they have **Don't Destroy With Owner** on (Step 3.7). Remove the player's entries from your own dictionaries and lists in the disconnect event.

7.4 Refusals. Return `Approved = false` and a short `Reason` such as `bad token`, `room full`, `token expired`. The client reads `NetworkManager.DisconnectReason`.

7.5 Reconnect (browser tabs drop).

- Hidden tabs slow down; a long hidden tab may lose the connection. Keep scripts tolerant: do not kick a player for one missed message.
- To reconnect: call `NetworkManager.Shutdown()`, wait until `ShutdownInProgress` is false, ask the lobby for a **new token**, then `StartClient()` again. The old player object is gone, so keep durable data (score) keyed by something you control, not by the client id.

CHECK IT WORKED

Join with two windows: both see two players. Close one: the server log shows `ClientDisconnected` and the other window loses that player object.

WATCH OUT

Common mistakes: keeping per-player data keyed by client id across reconnects (the id changes); forgetting to clean dictionaries; assuming `OnClientConnectedCallback` fires on every client for every other client (it does not: use `PeerConnected`).

NEXT → Step 8 shows how to test all of this quickly, and how to read the errors.

C PLAYERS, TESTING, PRACTICE

STEP 8 of 9 Test and debug



Why this step: A VM round trip (build, upload, publish, two windows) takes minutes. Most script mistakes can be found in seconds if you test in the Editor first and know what the error messages mean.

NEEDS

- Steps 2-7: scripts to test.

→ THIS STEP →

GIVES

- A fast Editor loop, a habit for logging, and a table of error messages with their causes.

Do

8.1 The fast loop: host in the Editor. The repository has `DevHost.cs`. Add it to the `Net` object, press Play, press **H**. The Editor becomes **server and client at once** (a *host*). Your player appears and every script runs.

In host mode	Remember
`IsServer` and `IsOwner` are both true for your own player	Test the server path and the owner path together
`[Rpc(SendTo.NotServer)]` does **not** reach the host	Use `SendTo.ClientsAndHost` for broadcasts (same result on the dedicated server)
A host does not prove WebGL behaviour	Still test in the real stack before you trust it
Multiplayer Play Mode can add more Editor clients	Use it for two-player checks

A real WebGL player can never be a host. Host mode is a **development tool only**.

8.2 Log who is speaking.

```
string Who => $"[{(IsServer ? "S" : "C")}{NetworkManager.LocalClientId}]";   // S0 = server, C1, C2 = clients
Debug.Log($"{Who} score changed to {Score.Value}");
```

8.3 Error messages and what they mean. The first five were read in the source.

Message	Cause	Fix
`The NetworkBehaviour must be spawned before calling this method.`	An Rpc was called before the object spawned (for example in `Awake`)	Call it after `OnNetworkSpawn`
`This RPC can only be sent by its owner.`	`InvokePermission = Owner` and the caller does not own it	Call from the owner, or relax the permission and validate
`This RPC can only be sent by the server.`	`InvokePermission = Server`	Call from the server

Message	Cause	Fix
`This method requires a runtime-specified send target.`	A `SendTo.SpecifiedInParams` Rpc was called without a target	Pass `RpcTarget.Single(...)`
`Write permissions (Server) for this client instance is not allowed!`	A client wrote a server-owned NetworkVariable	Write on the server, or send an Rpc
`Rpc method must end with 'Rpc' suffix!`	Build error: method name	Rename it
An object never appears on clients	The prefab is not in the Network Prefabs List, or you did not call `Spawn()`	Step 3
The client is refused with `bad token`	The lobby token expired (60 s) or is for another room	Join again quickly

8.4 Seeing it with real latency. The unreliable part of a real run is the browser. Always repeat the main flow once in the full stack (tutorial Step 11) before you call a feature done.

CHECK IT WORKED

Host in the Editor, move the player, collect something, see the score change, and get no errors in the console.

WATCH OUT

Common mistakes: only testing as a host; forgetting that `NotServer` skips the host; reading a variable before spawn.

NEXT → Step 9 puts steps 2 to 8 into one small feature.

C PLAYERS, TESTING, PRACTICE

STEP 9 of 9 Worked example: collect coins



Why this step: Each step answered one question in isolation. This feature uses all of them once, so you can see how the pieces depend on each other.

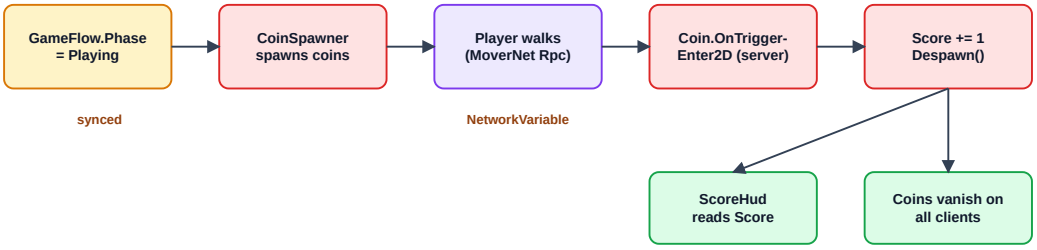
NEEDS

- Step 2: `GameFlow` as the match manager.
- Step 3: a coin prefab, spawned by the server.
- Step 4: `PlayerState.Score`.
- Step 6: private and shared reads.
- Step 8: host mode for testing.

→ THIS STEP →

GIVES

- A complete little loop: the match starts, coins appear, a player touches one, the score goes up on every screen.



The server decides every step. Clients only send input and read results.

Every step used once

Do

9.1 The loop and who does what.

Moment	Who runs it	Tool	File
Phase becomes `Playing`	server (after everyone is ready)	NetworkVariable `Phase` (Step 4)	`GameFlow.cs`
Coins appear	server	`InstantiateAndSpawn` (Step 3)	`CoinSpawner.cs`
A player touches a coin	**server** physics	`OnTriggerEnter2D`	`Coin.cs`
Score goes up	server writes	NetworkVariable `Score` (Step 4)	`PlayerState.cs`
Coin disappears everywhere	server	`Despawn` (Step 3)	`Coin.cs`
Score is shown	each client reads	`OnGUI` reading the variable (Step 6, pattern 1)	`ScoreHud.cs`

Notice what is missing: no Rpc is needed for touching a coin. The server already knows where the player is, because the server moves the player (Step 5), so the server detects the touch. A client never says "I got the coin".

9.2 Coin.cs (server decides).

```
public class Coin : NetworkBehaviour
{
    void OnTriggerEnter2D(Collider2D other)
    {
        if (!IsServer || !IsSpawned) return;                    // only the server decides
        if (!other.TryGetComponent(out PlayerState player)) return;
        player.AddScore(1);                                     // writes the NetworkVariable
        NetworkObject.Despawn();                                // removed on every machine
    }
}
```

9.3 CoinSpawner.cs (server spawns when the phase changes).

```
public override void OnNetworkSpawn()
{
    if (!IsServer) return;
    flow = GetComponent<GameFlow>();
    flow.Phase.OnValueChanged += OnPhase;               // react to the synced phase
}
void OnPhase(GamePhase previous, GamePhase next)
{
    if (next != GamePhase.Playing) return;
    for (int i = 0; i < count; i++)
        NetworkObject.InstantiateAndSpawn(coinPrefab, NetworkManager,
            position: new Vector3(Random.Range(-area, area), Random.Range(-area, area), 0f),
            rotation: Quaternion.identity);
}
```

9.4 ScoreHud.cs (clients only read). It finds the local player with `NetworkManager.LocalClient.PlayerObject` and shows `mine.Score.Value`. Nothing here writes.

9.5 Set it up.

1. Make a coin prefab: a sprite, **NetworkObject**, **CircleCollider2D** with *Is Trigger* on, and `Coin`. Add it to the Network Prefabs List.
2. Give the **player prefab** a **Rigidbody2D** with *Body Type: Kinematic*. Unity raises trigger events only if one side has a Rigidbody2D.
3. On the in-scene object that has `GameFlow` (and a NetworkObject) add `CoinSpawner` and drag the coin prefab into it.
4. Add `ScoreHud` to the `Net` object.
5. Host in the Editor (H). Call `GameFlow.SetReadyRpc(true)` for example from a test button; the phase becomes `Playing`, coins appear, walking into one raises the score.

9.6 Where each earlier question is answered.

Question (Step 1)	Answer in this example
When does it exist?	`CoinSpawner` spawns, `Coin` despawns
Who may change it?	Only the server: `AddScore`, `Despawn`
How do others see it?	`Score` is a NetworkVariable; coins are spawned objects; the player moves through NetworkTransform

Question (Step 1)	Answer in this example
How does a client ask?	By moving: `MoverNet` sends input with an Rpc
How do I get information?	`ScoreHud` reads the variable

CHECK IT WORKED

Two windows in the full stack: both see the coins, one player collects one, the coin disappears in both windows, and the score of that player changes in its own HUD.

WATCH OUT

Common mistakes: no Rigidbody2D on either side (no trigger events); the coin prefab not in the Network Prefabs List; `OnTriggerEnter2D` running on clients (guard with `IsServer`); writing `Score` from the client.

Quick reference

Names below were read in the NGO 2.13.3 source. Use it to look something up after you have read the steps.

I need to ...	Names	Where
Is this machine the server? the client? the owner?	`IsServer`, `IsClient`, `IsOwner`, `IsLocalPlayer`, `IsOwnedByServer`, `IsSpawned`	NetworkBehaviour
Who owns it, who am I?	`OwnerClientId`, `NetworkManager.LocalClientId`, `NetworkManager.ServerClientId` (0)	NetworkBehaviour, NetworkManager
Spawn and remove	`Spawn()`, `SpawnWithOwnership(id)`, `SpawnAsPlayerObject(id)`, `Despawn(destroy)`, `NetworkObject.InstantiateAndSpawn(...)`	NetworkObject
Ownership	`ChangeOwnership(id)`, `RemoveOwnership()`, `OnGainedOwnership()`, `OnLostOwnership()`	NetworkObject, NetworkBehaviour
Visibility	`NetworkShow(id)`, `NetworkHide(id)`, `SpawnWithObservers`	NetworkObject
Lifecycle	`OnNetworkSpawn`, `OnNetworkDespawn`, `OnNetworkPostSpawn`, `OnInSceneObjectsSpawned`	NetworkBehaviour
Synced values	`NetworkVariable<T>(value, readPerm, writePerm)`, `.Value`, `.OnValueChanged`, `NetworkList<T>.OnListChanged`	NetworkVariable
Remote calls	`[Rpc(SendTo.X, Delivery, InvokePermission, DeferLocal, AllowTargetOverride)]`, `RpcParams`, `RpcTarget.Single/Group/Not`	Messaging
Send targets	`Owner`, `NotOwner`, `Server`, `NotServer`, `Me`, `NotMe`, `Everyone`, `ClientsAndHost`, `Authority`, `NotAuthority`, `SpecifiedInParams`	RpcTarget.cs
Connection	`StartServer`, `StartClient`, `StartHost`, `Shutdown`, `ConnectionApprovalCallback`, `OnConnectionEvent`, `DisconnectReason`	NetworkManager
Lookups	`SpawnManager.SpawnedObjects`, `SpawnManager.GetPlayerNetworkObject(id)`, `LocalClient.PlayerObject`, `ConnectedClientsIds`, `NetworkObjectReference`	NetworkManager, SpawnManager
Time	`ServerTime.Time`, `LocalTime.Time`, tick rate in `NetworkConfig.TickRate`	NetworkManager
Custom messages (rare)	`CustomMessagingManager.SendNamedMessage`, `RegisterNamedMessageHandler`	CustomMessageManager.cs

Files in the repository

File	What it shows	Step
`unity-scripts/Net/ServerBootstrap.cs`	Server start, token approval, heartbeat	**2, 7**
`unity-scripts/Net/ClientConnect.cs`	Browser connects with room path and token	**2, 7**

`unity-scripts/Net/LobbyClient.cs`	On-screen Create / Join	**2**
`unity-scripts/Net/DevHost.cs`	Press H in the Editor to host	**8**
`unity-scripts/Gameplay/PlayerState.cs`	NetworkVariables and an owner-only Rpc	**4, 5**
`unity-scripts/Gameplay/MoverNet.cs`	Input Rpc, server moves the player	**5**
`unity-scripts/Gameplay/Chat.cs`	Client to server to everyone	**6**
`unity-scripts/Gameplay/GameFlow.cs`	Phase NetworkVariable, ready-up NetworkList	**2, 4, 9**
`unity-scripts/Gameplay/Coin.cs`	Server-side pickup	**9**
`unity-scripts/Gameplay/CoinSpawner.cs`	Server spawns when the phase changes	**9**
`unity-scripts/Gameplay/ScoreHud.cs`	Client reads and shows the score	**9**

How sure is each statement?

Source-checked: every NGO API name, default value and error message in this guide was read in the package source (tag v2.13.3) during this work. **Not run:** no Unity Editor was available, so none of the C# was compiled; the repository files are marked UNTESTED. **From memory:** the Mirror column of the translation table, and the behaviour of Unity's 2D trigger events (a Rigidbody2D must be on one of the objects).